

Coding & Solution

Date	25 September 2026
Team ID	6ab22227a614981d6d2c6451
Project Name	Import Data using Transform Maps
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements. .

Solution Summary

Field	Details
Repository Link / URL	https://github.com/Team-6ab22227a614981d6d2c6451/ServiceNow-Transform-Maps
Programming Language(s)	Rhino JavaScript, HTML, CSS
Framework(s) Used	ServiceNow Glide API, Transform Engine
Key Features Implemented	Automated Data Ingestion, Coalesce Deduplication, Analytics Dashboards, ACL Security
Pending / Incomplete Features	None
Setup / Run Instructions	1. Navigate to Load Data. 2. Upload Excel File.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

- 1.Ensured that the code is well-structured, commented, and follows best practices for your chosen technology stack.
- 2.The solution will address the core problem statement and implement the key functional requirements.
- 3.Included all source files, configuration files, and setup instructions needed to run the application.
- 4.Attached the code below.

1. Transform Map - onBefore Script

Purpose: This script runs automatically *before* the Transform Engine inserts or updates a record in your target table (u_employee_test). It validates the incoming Excel data to ensure data integrity and prevents corrupt rows from being processed.

javascript

```
(function runTransformScript(source, map, log, target /*undefined onInsert*/ ) {
  // 1. Mandatory Field Check: Ensure Employee ID exists before processing
  if (JSUtil.nil(source.u_employee_id)) {
    ignore = true; // Skip this row
    log.error("Import Failed: Employee ID is missing for record: " + source.u_name);
    return;
  }
  // 2. Data Cleaning: Validate Email Format using Regex
  var emailRegex = /^[a-zA-Z0-9._-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,4}$/;
  if (!JSUtil.nil(source.u_email) && !emailRegex.test(source.u_email)) {
    log.warn("Invalid email format for Employee ID " + source.u_employee_id + ". Proceeding with empty email.");
  }
}
```

```

        target.u_email = ""; // Clear invalid email to maintain database hygiene
    }
    // 3. Data Normalization: Standardize Department names to Uppercase
    if (!JSUtil.nil(source.u_department)) {
        target.u_department = source.u_department.toString().toUpperCase();
    }
})(source, map, log, target);

```

2. Field Map Script (Advanced Mapping)

Purpose: If a specific column in your Excel spreadsheet needs logical manipulation before hitting the target table, a Field Map script is used instead of a direct 1-to-1 map.

```

answer = (function transformEntry(source) {
    // If the Excel 'Location' column is empty, assign a default location
    if (JSUtil.nil(source.u_location)) {
        return "HQ - Default";
    }

    // Otherwise, return the exact location string provided in the spreadsheet
    return source.u_location.toString().trim();
})(source);

```

3. GlideRecord Validation Script (Background Script)

Purpose: After the Import Set is transformed, this script can be run by the System Admin to programmatically verify that the data (and the deduplication) was successfully stored in the custom target table without having to manually scroll through list views.

```

// Query the target table to validate the import
var empGr = new GlideAggregate('u_employee_test');
empGr.addAggregate('COUNT');
empGr.query();
var totalRecords = 0;
if (empGr.next()) {
    totalRecords = empGr.getAggregate('COUNT');
    gs.print("Total Employee Records Validated in Target Table: " + totalRecords);
}

// Check for recent updates to verify Coalesce functionality
var recentUpdatesGr = new GlideRecord('u_employee_test');
recentUpdatesGr.addEncodedQuery('sys_updated_onONToday@javascript:gs.beginningOfToday()@javascript:gs.endOfToday()');
recentUpdatesGr.query();
gs.print("Total Records Inserted or Updated Today: " + recentUpdatesGr.getRowCount());
if (totalRecords > 0) {
    gs.print("SUCCESS: Data Import and Transform Map Coalesce executed correctly.");
} else {
    gs.print("WARNING: No records found. Please check the 'u_employee_import' staging table for errors.");
}

```

